Competitive Programming - Code Snippets

6 file(s) · generated 2026-11-05

teoria_numeros_general.cpp — 205 lines

```cpp
#include<bits/stdc++.h>
using namespace std;
//________tener rapido la factorizacion en primos de muchos numeros hasta
2e5____________________
const int MAXN = 1e6+1;//este lo podes cambiar
int min_div[MAXN];
void div(){
  min_div[1] = 1;
  for(int i = 2; i <MAXN; i++){
    if(min_div[i]==0){
      for(int j = i; j < MAXN; j+=i)min_div[j] = i;
    }
  }
}

void factorizar(int x, vector<int> &factores){
//x es el numero a factorizar y factores es el array donde los guarda.
//para usarlo NO OLVIDES INICIALIZAR DIV()
  if(x == 1){
    factores.push_back(1);
    return;
  }
  int i = min_div[x];
  while(x%i == 0){
    factores.push_back(i);
    x/= i;
  }
  factorizar(x, factores);
}
//____________numero de divisores de muchos numeros hasta 2e5
const int MAXN = 1e6+1;//este lo podes cambiar
int min_div[MAXN];

void div(){
  min_div[1] = 1;
  for(int i = 2; i <MAXN; i++){
    if(min_div[i]==0){
      for(int j = i; j < MAXN; j+=i)min_div[j] = i;
    }
  }
}
void count_divs(int x, int &count){
//cout << x << endl;
int r = 0;
if(x == 1){
return;
}
int i = min_div[x];
```

```
while(x%i == 0){
r++;
x/= i;
}
count*=(r+1);
count_divs(x, count);
}
//__________MCM de un array
long long mcm_array(vector<int> v, int MOD){
//MCM de muchos numeros (cada numero menor a 2e5).
//MOD es un modulo pues mcm puede ser muy grande. Usa: factorizar
long long ans = 1;
22
map<int,int> m;
vector<int> factores(0);
for(auto k: v){
factores.resize(0);
factorizar(k, factores);
int j = factores[0];
int r = 1;
for(int i = 1; i<int(factores.size()); i++){
if(factores[i] == factores[i-1])r++;
else m[factores[i-1]] = max(m[factores[i-1]], r), r = 1;
}
int s = int(factores.size())-1;
m[factores[s]] = max(m[factores[s]], r);
}
ans = 1;
for(auto k:m){
long long g = pow(k.first,k.second);
ans = (ans*g)%MOD;
}
return ans;
}
//_________________
//________FUNCION PHI. PHI(N)= CANTIDA DE COPRIMOS ANTES QUE N
//SIRVE PARA HALLAR INVERSO MODULAR POR EJEMPLO,
//PUES SI A, M COPRIMOS(UNICO CASO EN QUE EXISTE EL INVERSO)
//(A^[PHI(M)-1]) = A^(-1)(MOD M)
//OJO QUE SI M PRIMO ALCANZA CON A^(M-1) PUES PHI(PRIMO)=PRIMO-1
long long phi_to_n(long long n) {
//da la funcion phi de un numero en o(sqrt(n)) sin buscar su factorizacion
long long result = n;
for (int i = 2; i * i <= n; i++) {
if (n % i == 0) {
while (n % i == 0)
n /= i;
result -= result / i;
}
}
if (n > 1)
result -= result / n;
return result;
}
```

```cpp
void phi_1_to_n(int n, vector<int> &ans){//phi de {1,...,n} en nlogn
vector<int> phi(n + 1);
phi[0] = 0;
phi[1] = 1;
for (int i = 2; i <= n; i++)phi[i] = i - 1;
for (int i = 2; i <= n; i++){
for (int j = 2 * i; j <= n; j += i)phi[j] -= phi[i];
}
ans = phi;
}
//_______ALGORTIMO DE EUCLUIDES PARA ENCONTRAR SOLUCONES DE A*X+B*Y = gcd(A,B)
//(TARDA LO MISMO QUE GCD)
long long extenden_euclidean(long long a, long long b, long long& x, long long& y) {
if (b == 0) {
x = 1;
23
y = 0;
return a;
}
long long x1, y1;
long long d = extenden_euclidean(b, a % b, x1, y1);
x = y1;
y = x1 - y1 * (a / b);
return d;
}
//________usa el algoritmo de euclides anterior para decidir si existe solucion
//(y hallarla en x, y) de ax +by = c
bool find_any_solution(
long long a, long long b, long long c,
long long &x0, long long &y0, long long &g
) {
//x, y no necesitanm tener valores inicales, en ellos se guardará la solucion.
g = extenden_euclidean(abs(a), abs(b), x0, y0);
if (c % g) {
return false;
}
x0 *= c / g;
y0 *= c / g;
if (a < 0) x0 = -x0;
if (b < 0) y0 = -y0;
return true;
}
//____________INVERSO MODULAR CON ALGORITMO DE EUCLIDES
//(NO NECESITA PHI NI FACTORIZACION).
//IGUAL DE RAPIDO QUE GCD (ojo que despues hay uno para calcular
//el inverso de un array de tamaño n en o(n)
//DADOS A, M COPRIMOS, NOS DA INVERSO DE A MOD M.
int inverso_modm(long long a, long long m){
//devuelve inverso de a(positivo) si son coprimos,
//-1 en otro caso(si no son coprimos no existe el inverso de a)
a = a%m;
long long x, y;//sin valores
int g = extenden_euclidean(a, m, x, y);//este me da ax+my=gcd(a,m) = 1
if(g!= 1) return -1;
```

```
return ((x%m+m)%m);
}
//dado m primo devuelve el inverso de [1,..., m-1] en o(m)
void inverso_to_1_m(long long m, vector<long long> & inv){
//el problema aca es si m es muy grande. para eso sirve el algortimo que sigue a este
inv.resize(m+1);
inv[1] = 1;
for(int a = 2; a < m; ++a) inv[a] = m - (long long)(m/a) * inv[m%a] % m;
}
//dado un array de tamaño n nos da todos sus inversos modulo m
//(no necesariamente primo, posiblemente grande) en o(n)
void inverso_to_array(vector<long long> v, long long m, vector<long long> & ans){
//v es el array original. en ans se guardan sus inversos.
//se asume que todos tienen inversos(que son coprimos con m)
long long n = v.size();
24
long long producto = 1;
vector<long long> prefix(n);
vector<long long> sufix(n);
for(auto &i : v) producto = (producto*i)%m;
long long x, y;
long long d = extenden_euclidean(producto, m, x, y);
long long inv = (x%m+m)%m;//este es el inversol producto de todos
prefix[0] = 1;
sufix[n-1] = 1;
for(int i = 1; i < n; i++){
prefix[i] = (prefix[i-1]*v[i-1])%m;
sufix[n-i-1] = (sufix[n-i]*v[n-i])%m;
}
for(int i = 0; i < n; i++){
ans[i] = (((prefix[i]*sufix[i])%m)*inv)%m;
}
}
// pow(a, b) % MOD.
// fomra mas facil de hallar eñ inverso modular a respecto de un primo p es mpow(a,p-
2,MOD)
long long mpow(long long a, long long b, long long MOD) {
long long x = 1;
while (b > 0) {
if (b & 1) { /* fast way of checking if b is odd */
x = (x * a) % MOD;
}
a = (a * a) % MOD; /* compute the next power of 2 */
b >>= 1; /* this is equivalent to b = floor(b / 2) */
}
return x;
}
int main(){
int n; cin >> n;
cout << phi_to_n(n);
}
```

combinatoria_general.cpp — 33 lines

```cpp
#include<bits/stdc++.h>
using namespace std;
//Propiedades Basicas de numeros combinatorios
C(n,k) = C(n-1,k-1) + C(n-1,k)
C(n,k) = C(n,n-k)
//________calcular cualquier combinatorio C(a,b)
//con 0<=b<=a<=1e6 en o(1) con precomputo de O(n+logMOD) creo
//usa mpow que está en teoria de numeros
//NO OLVIDES INICIALIZAR PRECOMPUTO()
const long long MOD = 1e9+7;//ojo este
long long MAXN = 1e6+100;


vector<long long> factorial(MAXN+1);
vector<long long> inversos(MAXN+1);
void precomputo(){

  factorial[0] = 1;
  for(int i =1; i <= MAXN; i++){
    factorial[i] = (factorial[i-1]*i)%MOD;
  }

  inversos[MAXN] = mpow(factorial[MAXN], MOD-2, MOD);
  for(int i = MAXN-1; i >=0; i--){
    inversos[i] = (inversos[i+1]*(i+1))%MOD;
  }
}
long long combinatorio(long long a, long long b){
  return (
      (factorial[a]*inversos[b]%MOD)*
      inversos[a-b])%MOD;
}
```

Geometria_mejorado.cpp — 300 lines

```cpp
#include <bits/stdc++.h>

using namespace std;

typedef complex<long double> point;
#define ld long double
#define x real()
#define y imag()
const ld EPS = 1e-9;
const ld PI = acos(-1.0L);

// _________ FUNCIONES AUXILIARES DE PRECISIÓN _________
// Retorna 1 si es positivo, -1 si negativo, 0 si es cercano a cero
int sign(ld a) {
    return (a > EPS) - (a < -EPS);
}

// _________ OPERACIONES FUNDAMENTALES _________
// Producto punto: proyecta un vector sobre otro. (<0)
ld dot(point a, point b) { return (conj(a) * b).x; }

// Producto cruz: área orientada (me dice >0 si a esta a la izquierda del vector (0,b);
<0 "a" a la derecha de b; == 0 misma recta )
ld cross(point a, point b) { return (conj(a) * b).y; }

// Posición relativa de 'p' respecto a la recta dirigida a->b
// 1: izquierda, -1: derecha, 0: colineales
int pos_relativa(point p, point a, point b) {
    return sign(cross(b - a, p - a));
}

// _____________________ DISTANCIAS Y PROYECCIONES de un punto a una recta o
segmento_____________________ _________
// Devuelve la proyección ortogonal del punto p sobre la recta [a, b]
point proyeccion_punto_recta(point p, point a, point b) {
    point dir = b - a;
    return a + dir * (dot(p - a, dir) / norm(dir)); // norm(dir) equivale a |dir|^2
}

// Distancia del punto p a la recta infinita [a, b]
ld dist_punto_recta(point p, point a, point b) {
    return abs(cross(b - a, p - a)) / abs(b - a);
}

// DEVUELVE TRUE SI EL PUNTO P ESTÁ EN EL SEGMENTO [a, b]
bool point_in_segment(point p, point a, point b) {
    // Debe ser colineal Y el producto punto hacia los extremos debe ser <= 0
    return pos_relativa(p, a, b) == 0 && sign(dot(a - p, b - p)) <= 0;
}

// Distancia del punto p al segmento [a, b]
ld dist(point p, point a, point b) {
```

```
    if (a == b) return abs(p - a);
    if (sign(dot(b - a, p - a)) < 0) return abs(p - a); // Cae fuera del lado de a
    if (sign(dot(a - b, p - b)) < 0) return abs(p - b); // Cae fuera del lado de b
    return dist_punto_recta(p, a, b);                   // Cae en el medio, devolvemos
dist a recta
}

// _____________________ POLÍGONOS ___________________________________
// DISTANCIA DE UN PUNTO A UN POLIGONO /O(cant vertices)
//distancia dentre dos poligonos se hace con minkowski y aparece mas abajo
ld dist_point_poligon(point p, const vector<point> &poligono) {
    ld mini = LDBL_MAX;
    int n = poligono.size();
    for(int i = 0; i < n; i++) {
        mini = min(mini, dist(p, poligono[i], poligono[(i + 1) % n]));
    }
    return mini;
}

// Área del polígono. O(cant vertices)
ld area(const vector<point> &poligono) {
    ld ans = 0;
    int n = poligono.size();
    for(int i = 0; i < n; i++) {
        ans += cross(poligono[i], poligono[(i + 1) % n]);
    }
    return abs(ans) / 2.0;
}

// Posición de un punto respecto a polígono: 0 si está en el borde, 1 adentro, -1
afuera(O(cant vertices))
//OJO QUE SI EL PROBLEMA ES "TE DOY MUCHOS PUNTOS Y DECIME SI CAEN DENTRO" hay una
solucion con consultas de logN mas abajo
int in_polygon(point p, const vector<point> &poligono) {
    bool inside = false;
    int n = poligono.size();
    for (int i = 0; i < n; i++) {
        point a = poligono[i], b = poligono[(i + 1) % n];
        if (point_in_segment(p, a, b)) return 0; // Borde
        if ((a.y > p.y) != (b.y > p.y)) {
            ld intersect_x = a.x + (p.y - a.y) * (b.x - a.x) / (b.y - a.y);
            if (p.x < intersect_x) inside = !inside;
        }
    }
    return inside ? 1 : -1;
}

// _________ _______________INTERSECCIONES de rectas y
segmentos_____________________________
// Intersección entre dos rectas [a,b] y [p,q]
point intersection(point a, point b, point p, point q) {
    ld c1 = cross(p - a, b - a);
    ld c2 = cross(q - a, b - a);
    return (c1 * q - c2 * p) / (c1 - c2); // Nota: si son paralelas, c1 == c2 y explota
```

```
}

// ¿Dos segmentos se cortan? (Corregido para detectar uniones en forma de T y
colineales)
bool interseccion_segmentos(point a, point b, point p, point q) {
    int oa = pos_relativa(a, p, q), ob = pos_relativa(b, p, q);
    int op = pos_relativa(p, a, b), oq = pos_relativa(q, a, b);

    // Se cruzan en un único punto estrictamente
    if (oa != ob && op != oq) return true;

    // Casos borde: colineales superpuestos o un extremo toca al otro segmento
    if (point_in_segment(a, p, q) || point_in_segment(b, p, q) ||
        point_in_segment(p, a, b) || point_in_segment(q, a, b)) return true;

    return false;
}

//interseccion de dos poligonos esta en poligonos tryhard

// _____________________ INTERSECCIONES DE CIRCUNFERENCIAS_____________________

// Intersección entre circunferencia C(c, r) y la recta infinita que pasa por a y b
vector<point> intersection_circle_line(point c, ld r, point a, point b) {
    point p = proyeccion_punto_recta(c, a, b);
    ld d = abs(c - p);

    if (d > r + EPS) return {}; // No se cruzan (0 puntos)
    if (abs(r - d) <= EPS) return {p}; // Tangente (1 punto)

    ld h = sqrtl(r * r - d * d);
    point dir = (b - a) / abs(b - a);
    return {p - dir * h, p + dir * h}; // Secante (2 puntos)
}

// Intersección de dos circunferencias C1(c1, r1) y C2(c2, r2)
vector<point> intersection_circle_circle(point c1, ld r1, point c2, ld r2) {
    ld d = abs(c2 - c1);

    // Disjuntos, uno dentro del otro, o circunferencias idénticas/concéntricas
    if (d > r1 + r2 + EPS || d < abs(r1 - r2) - EPS || d < EPS) return {};
    ld a = (r1 * r1 - r2 * r2 + d * d) / (2.0L * d);
    ld h = sqrtl(max(0.0L, r1 * r1 - a * a));
    point p = c1 + (c2 - c1) * (a / d);
    if (h <= EPS) return {p}; // Tangentes (1 punto)
    point offset = (c2 - c1) * point(0, 1) * (h / d);
    return {p + offset, p - offset}; // Secantes (2 puntos)
}


// _____________________________________ ÁNGULOS
_____________________________________________
ld angle(point a, point b) { // Ángulo del vector a->b
    return arg(b - a);
```

```
}

// Ángulo entre a, b y c. Devuelve el ángulo en (-pi, pi]
ld angle_ABC(point a, point b, point c) {
    ld g = arg(a - b) - arg(c - b);
    if (g <= -PI) g += 2.0L * PI;
    if (g > PI) g -= 2.0L * PI;
    return g;
}

//_____________________________________poligonos
tryhard_____________________________________________

// Función auxiliar para ordenar puntos (Izquierda a Derecha, Abajo a Arriba)
bool compP(const point& a, const point& b) {
    if (abs(a.x - b.x) > EPS) return a.x < b.x;
    return a.y < b.y;
}

vector<point> convex_hull(vector<point>& pts) {//dado un conjunto de puntos, cual es el
poligono mas pequeño que los encierra(ejemplo para encontrar los dos mas lejanos)
    int n = pts.size(), k = 0;
    if (n <= 2) return pts;
    vector<point> h(2 * n);
    sort(pts.begin(), pts.end(), compP);
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && cross(h[k-1] - h[k-2], pts[i] - h[k-2]) <= 0) k--;
        h[k++] = pts[i];
    }

    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(h[k-1] - h[k-2], pts[i] - h[k-2]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}

//__________ distancia máxima al cuadrado entre dos puntos del conjunto en NlogN_____
ld max_distance(vector<point>& pts) {
    vector<point> h = convex_hull(pts);
    int n = h.size();

    if (n <= 1) return 0;
    if (n == 2) return norm(h[0] - h[1]);
    ld max_dist2 = 0;
    int j = 1;
    for (int i = 0; i < n; i++) {
        int next_i = (i + 1) % n;
        while (abs(cross(h[next_i] - h[i], h[(j + 1) % n] - h[i])) >
               abs(cross(h[next_i] - h[i], h[j] - h[i]))) {
            j = (j + 1) % n;
        }
        max_dist2 = max({max_dist2, norm(h[i] - h[j]), norm(h[next_i] - h[j])});
```

```
    }
    return max_dist2; // O usa sqrtl(max_dist2) si te piden la distancia lineal real
}



// __________O(log N) punto está dentro de un polígono CONVEXO_________________
// Retorna: 1 (adentro), 0 (borde), -1 (afuera)
//PRIMERO va esto para preparar el poligono. VA EN TU MAIN DESPUES DE LEER EL
POLIGONO(poly)
//copiar de aca
if (pos_relativa(poly[2], poly[0], poly[1]) < 0) {
    reverse(poly.begin(), poly.end());
}
auto it = min_element(poly.begin(), poly.end(), compP);
rotate(poly.begin(), it, poly.end());
//hasta aca y ahora si la funcion:

int in_convex_polygon_log(point p, const vector<point>& poly) {//usando lo que dice aca
arriba,
    int n = poly.size();
    if (n < 3) {
        if (n == 1) return (abs(p - poly[0]) <= EPS) ? 0 : -1;
        if (n == 2) return point_in_segment(p, poly[0], poly[1]) ? 0 : -1;
        return -1;
    }
    point p0 = poly[0];
    int pos1 = pos_relativa(p, p0, poly[1]);
    int posn = pos_relativa(p, p0, poly[n - 1]);
    if (pos1 < 0 || posn > 0) return -1;

    if (point_in_segment(p, p0, poly[1])) return 0;
    if (point_in_segment(p, p0, poly[n - 1])) return 0;
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int mid = l + (r - l) / 2;
        if (pos_relativa(p, p0, poly[mid]) >= 0) {
            l = mid; // El punto está a la izquierda (o sobre) la diagonal p0 ->
poly[mid]
        } else {
            r = mid; // El punto está a la derecha
        }
    }
    int final_pos = pos_relativa(p, poly[l], poly[r]);

    if (final_pos > 0) return 1; // Estrictamente adentro
    if (final_pos == 0 && point_in_segment(p, poly[l], poly[r])) return 0; // Sobre la
arista exterior de la tapa

    return -1; // Afuera
}

//_________________SUMA DE MINKOWSKI en O(|P|+|Q|): ____________________
```

```cpp
//dados dos poligonos convexos, sumar los poligonos(que tambien da convexo)
//sirve para: 1) si deducis que un punto debe ser comb lineal de los puntos de varios poligonos, haes la suma y verificas si cae dentro.
//2) ver si dos poligonos se intersectan en O(|P|+|Q|) (chequeas si (0,0) esta en P-Q)
//primero esto fuera del main
void reorder_polygon(vector<point>& P) {
    auto it = min_element(P.begin(), P.end(), compP);
    rotate(P.begin(), it, P.end());
}
//va todo esto en el main antes de llamar a la suma
if (pos_relativa(P[2], P[0], P[1]) < 0) reverse(P.begin(), P.end());

if (pos_relativa(Q[2], Q[0], Q[1]) < 0) reverse(Q.begin(), Q.end());

//algoritmo:
// Suma de Minkowski en O(N + M)
vector<point> minkowski(vector<point> P, vector<point> Q) {
    reorder_polygon(P);
    reorder_polygon(Q);
    P.push_back(P[0]); P.push_back(P[1]);
    Q.push_back(Q[0]); Q.push_back(Q[1]);

    vector<point> result;
    size_t i = 0, j = 0;

    while (i < P.size() - 2 || j < Q.size() - 2) {
        result.push_back(P[i] + Q[j]);
        point edgeP = P[i + 1] - P[i];
        point edgeQ = Q[j + 1] - Q[j];

        int c = sign(cross(edgeP, edgeQ));

        if (c >= 0 && i < P.size() - 2) ++i;
        if (c <= 0 && j < Q.size() - 2) ++j;
    }

    return result; // El polígono resultante ya sale convexo y ordenado
}



int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout << setprecision(20);
    return 0;
}
```

complete_notes.cpp — 37 lines

```
# Bellman Ford

void solve()
{
  vector<int> d(n, INF);
  d[v] = 0;
  vector<int> p(n, -1);
  int x;
  for (int i = 0; i < n; ++i) {
  x = -1;
  for (Edge e : edges)
  if (d[e.a] < INF)
  1
  if (d[e.b] > d[e.a] + e.cost) {
  d[e.b] = max(-INF, d[e.a] + e.cost);
  p[e.b] = e.a;
  x = e.b;
  }
  }
  if (x == -1)
  cout << "No negative cycle from " << v;
  else {
  int y = x;
  for (int i = 0; i < n; ++i)
  y = p[y];
  vector<int> path;
  for (int cur = y;; cur = p[cur]) {
  path.push_back(cur);
  if (cur == y && path.size() > 1)
  break;
  }
  reverse(path.begin(), path.end());
  cout << "Negative cycle: ";
  for (int u : path)
    cout << u << ' ';
  }
}
```

notes/snippet.py — 400 lines

```python
# ============================================================================
# Python cheat sheet for competitive programming
# ----------------------------------------------------------------------------
# Everything here is plain Python 3. Copy the pieces you need.
# The file itself is valid Python:  python3 snippet.py
# ============================================================================

import sys
import math
import heapq
import bisect
from collections import deque, Counter, defaultdict
from functools import lru_cache

# ----------------------------------------------------------------------------
# 0. BIG INTEGERS - read this first
# ----------------------------------------------------------------------------
# Python ints have UNLIMITED size. No overflow, ever:
#   2**1000, 10**100000, factorial(10000) are all fine.
#
# The only trap: Python 3.11+ limits int <-> str conversion to 4300
# digits by default. A 5000-digit input/output will crash unless you
# disable the limit BEFORE reading/printing. Keep these two lines:

try:
    sys.set_int_max_str_digits(0)   # 0 = no limit
except AttributeError:
    pass                            # Python < 3.11 has no limit

# Huge-number operations (all exact):
#   a + b, a - b, a * b
#   a // b        integer division (floor)
#   a % b         remainder
#   divmod(a, b)  -> (a // b, a % b)   (one division, faster)
#   a ** b        power, a ** 0.5 is FLOAT (don't use for big a)
#   pow(a, b, mod) fast modular exponentiation
#   math.isqrt(a) integer square root, exact
#   a.bit_length()
#   int(s)        parse from string:  int("123...")   (any length)
#   str(a)        convert to string:  len(str(a)) = digit count
#   int(s, 16)    parse base 16;  int("ff", 16) == 255
#   bin(a), hex(a), f"{a:b}"  -> strings with/without prefix
#   int(str(a)[::-1])         reverse the digits
#   sum(map(int, str(a)))     digit sum
#
# NEVER use `/` on big integers: it returns a float and loses precision.
#   a / b      -> float   WRONG for big numbers
#   a // b     -> exact int  RIGHT
#
# Decimal is only for decimal fractions; its default precision is 28
# digits, so never use it to store a huge integer:
#   from decimal import Decimal, getcontext
```

```
#   getcontext().prec = 100000        # if you really need it
#
# ------------------------------------------------------------------------
# 1. INPUT
# ------------------------------------------------------------------------
# Fastest and simplest: read the WHOLE input, split on any whitespace.
# Works whether the values are on one line or many lines.
#
#   data = sys.stdin.buffer.read().split()   # list of bytes
#   a = int(data[0])                          # int() accepts bytes too
#   b = int(data[1])
#
# If you need strings instead of bytes:
#   data = sys.stdin.read().split()           # list of str
#
# Read one line at a time:
#   n = int(sys.stdin.readline())
#   a, b = map(int, sys.stdin.readline().split())
#   arr = list(map(int, sys.stdin.readline().split()))
#
# Read n numbers, one per line:
#   arr = [int(sys.stdin.readline()) for _ in range(n)]
#
# Read n lines of two numbers:
#   pairs = [tuple(map(int, sys.stdin.readline().split())) for _ in range(n)]
#
# Tip: target problems usually put the count first, e.g.:
#   n = int(data[0]); arr = list(map(int, data[1:1+n]))
#
# ------------------------------------------------------------------------
# 2. OUTPUT
# ------------------------------------------------------------------------
# print() puts a space between arguments and a newline at the end:
#   print(a, b)              ->  "12 34\n"
#   print(a, b, sep=", ")    ->  "12, 34\n"
#   print(a, end=" ")        ->  no newline
#   print(*arr)              ->  space-separated elements
#
# Fast output for many lines (build one string, write once):
#   out = []
#   out.append(str(x))
#   sys.stdout.write("\n".join(out))
#
# Or: sys.stdout.write(" ".join(map(str, arr)))
#
# ------------------------------------------------------------------------
# 3. VARIABLES, TYPES, OPERATORS
# ------------------------------------------------------------------------
#   x = 10            int
#   y = 10.5          float
#   s = "abc"         str
#   ok = True         bool
#   z = None          "nothing"
#
```

```
# Arithmetic:  +  -  *  /  //  %  **
#   Careful:  7 / 2 = 3.5     7 // 2 = 3
#             -7 // 2 = -4    (floor, NOT truncation toward zero!)
#             -7 % 2 = 1      (result has sign of divisor)
#   For C-like truncation on negatives, use int(a / b) only on small ints,
#   or: q = a // b; if (a % b != 0) and ((a < 0) != (b < 0)): q += 1
#
# Comparison:  ==  !=  <  <=  >  >=
# Boolean:     and  or  not
# Bitwise:     &  |  ^  ~  <<  >>     (also work on big ints)
# Identity:    is / is not           (use == for numbers/strings)
#
# Chained comparison is allowed:  0 <= x < n
# Assignment shortcuts:  x += 1   x -= 1   x *= 2   x //= 2
#
# ---------------------------------------------------------------------
# 4. CONDITIONALS
# ---------------------------------------------------------------------
# if x > 0:
#     ...
# elif x == 0:
#     ...
# else:
#     ...
#
# One-line:   sign = -1 if x < 0 else 0 if x == 0 else 1
# No switch statement. Use if/elif or a dict of functions.
#
# ---------------------------------------------------------------------
# 5. LOOPS
# ---------------------------------------------------------------------
# while condition:
#     ...
#
# for i in range(n):          # 0, 1, ..., n-1
# for i in range(1, n + 1):   # 1 .. n
# for i in range(5, 15, 2):   # 5, 7, 9, 11, 13
# for i in range(n - 1, -1, -1):  # n-1 down to 0  (reverse loop)
#
# for x in arr:               # iterate values
# for i, x in enumerate(arr): # index + value
# for i, x in enumerate(arr, start=1):   # 1-based index
# for a, b in zip(A, B):      # walk two lists together
#
# break     leaves the loop
# continue  skips to the next iteration
# pass      does nothing
#
# for/while can have an `else` that runs only if the loop did NOT break:
# for x in arr:
#     if x == target:
#         break
# else:
#     print("not found")      # reached only when nothing matched
```

```
#
# Python has no do-while. Emulate it:
# while True:
#     ...
#     if condition:
#         break
#
# ---------------------------------------------------------------------
# 6. FUNCTIONS
# ---------------------------------------------------------------------
# def add(a, b=0):        # b has a default value
#     return a + b
#
# def min_max(arr):
#     return min(arr), max(arr)     # returns a tuple
#
# lo, hi = min_max([3, 1, 2])
#
# Recursion: raise the limit first (default is 1000)!
# sys.setrecursionlimit(1_000_000)
#
# Memoization (top-down DP) in one line:
# @lru_cache(maxsize=None)
# def fib(n):
#     if n < 2:
#         return n
#     return fib(n - 1) + fib(n - 2)
#
# Nested functions and closures work normally. `global` is needed only
# when a function assigns to a module-level variable.
#
# ---------------------------------------------------------------------
# 7. LISTS (dynamic arrays)
# ---------------------------------------------------------------------
# a = []                  empty
# a = [1, 2, 3]
# a = [0] * n             n zeros
# a = list(map(int, data))
#
# a.append(x)             push back
# a.pop()                 remove and return last
# a.pop(i)                remove and return index i
# a.insert(i, x)          insert at i
# a.remove(x)             remove first value equal to x
# a.extend(b)             append all of b
# a.index(x)              first position of x
# a.count(x)              occurrences
# len(a)                  size
# x in a                  membership (O(n))
# a.sort()                in-place sort
# a.sort(reverse=True)
# a.reverse()
# b = sorted(a)           returns a new sorted list
# sum(a), min(a), max(a)
```

```
#
# Indexing: a[0] first, a[-1] last.
# Slicing:  a[l:r] is indices l .. r-1 (r exclusive)
#           a[:k] first k, a[k:] from k, a[::-1] reversed copy
#           slicing always makes a copy
#
# List comprehension (very common):
# b = [x * x for x in a]
# b = [x for x in a if x > 0]
# b = [x * 2 if x > 0 else 0 for x in a]
#
# 2D grid - WRITE IT THIS WAY:
# grid = [[0] * m for _ in range(n)]
# NEVER: grid = [[0] * m] * n     (all rows are the SAME object)
#
# ---------------------------------------------------------------------
# 8. STRINGS
# ---------------------------------------------------------------------
# Strings are immutable: s[i] = 'x' is an error. Build a list and join:
#   chars = list(s); chars[i] = 'x'; s = "".join(chars)
#
# s[i], len(s), s[l:r], s[::-1]     (reverse)
# s.upper(), s.lower()
# s.split()                 split on whitespace
# s.split(",")              split on comma
# " ".join(arr)             join list of strings
# s.strip()                 remove whitespace at both ends
# s.startswith(p), s.endswith(p)
# s.find(p)                 index or -1
# s.replace(old, new)
# s.isdigit(), s.isalpha(), s.isupper()
# ord(ch) -> code, chr(code) -> char
# Sorting characters: "".join(sorted(s))
#
# Comparing strings: s1 < s2 is lexicographic.
# Input like "abc" is already a string; single chars compare fine.
#
# ---------------------------------------------------------------------
# 9. DICTS AND SETS
# ---------------------------------------------------------------------
# d = {}                    empty dict
# d[key] = value
# d[key]                    KeyError if missing
# d.get(key, default)       safe read
# key in d                  membership (O(1))
# del d[key]
# d.keys(), d.values(), d.items()
# for k, v in d.items(): ...
#
# Counter counts things:
# c = Counter(arr)
# c[x], c.most_common(3)
#
# defaultdict gives missing keys a default (no KeyError):
```

```
# g = defaultdict(list)
# g[u].append(v)
# cnt = defaultdict(int)
# cnt[x] += 1
#
# Sets:
# s = set()
# s.add(x), s.discard(x), x in s
# A | B union, A & B intersection, A - B difference
#
# ------------------------------------------------------------------------
# 10. SORTING / SEARCHING
# ------------------------------------------------------------------------
# Sort tuples by first, then second:
# arr.sort(key=lambda p: (p[0], p[1]))
# Sort by first ascending, second descending:
# arr.sort(key=lambda p: (p[0], -p[1]))
# Sort indices by value:
# order = sorted(range(n), key=lambda i: arr[i])
#
# Binary search on a SORTED list (bisect):
# i = bisect.bisect_left(a, x)    # first index with a[i] >= x
# j = bisect.bisect_right(a, x)   # first index with a[i] > x
# count of x = j - i
# Insert keeping sorted: bisect.insort(a, x)
#
# Manual binary search:
# lo, hi = 0, n - 1
# while lo <= hi:
#     mid = (lo + hi) // 2
#     if a[mid] == x:
#         break
#     elif a[mid] < x:
#         lo = mid + 1
#     else:
#         hi = mid - 1
#
# ------------------------------------------------------------------------
# 11. USEFUL MODULES
# ------------------------------------------------------------------------
# math.gcd(a, b), math.lcm(a, b)
# math.isqrt(n)            exact floor(sqrt(n)), use for big ints
# math.inf                 infinity
# heapq: min-heap
#   h = []
#   heapq.heappush(h, x)
#   x = heapq.heappop(h)
#   heapq.heapify(arr)
#   max-heap: push -x
# deque: O(1) pops from both ends (use for BFS)
#   q = deque([start])
#   q.append(x); q.popleft()
#   q.appendleft(x); q.pop()
#
```

```python
# ---------------------------------------------------------------------
# 12. TEMPLATE: solve one test case
# ---------------------------------------------------------------------
def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    arr = list(map(int, data[1:1 + n]))
    # ... your algorithm ...
    print(sum(arr))


# Template: handle T test cases first.
def solve_many():
    data = sys.stdin.buffer.read().split()
    t = int(data[0])
    pos = 1
    out = []
    for _ in range(t):
        n = int(data[pos])
        pos += 1
        arr = list(map(int, data[pos:pos + n]))
        pos += n
        out.append(str(sum(arr)))
    sys.stdout.write("\n".join(out))


# ---------------------------------------------------------------------
# 13. TEMPLATE: huge number split (the f.py problem)
# ---------------------------------------------------------------------
# Given huge a and c, find m such that:
#   right = a % 10**m, left = a // 10**m, left // right == c
# Returns (left, right) or None.
def big_split(a, c):
    lo, hi = 1, len(str(a)) - 1
    while lo <= hi:
        m = (lo + hi) // 2
        p = 10 ** m
        left, right = divmod(a, p)
        if right == 0:
            lo = m + 1
            continue
        q = left // right
        if q == c:
            return left, right
        if q < c:          # need bigger quotient -> smaller m
            hi = m - 1
        else:              # need smaller quotient -> bigger m
            lo = m + 1
    return None


# ---------------------------------------------------------------------
# 14. GOTCHAS CHECKLIST
# ---------------------------------------------------------------------
```

```python
# [ ] Set sys.set_int_max_str_digits(0) before huge int read/print.
# [ ] Use // not / (never mix float with big ints).
# [ ] Python // floors toward -inf; C truncates toward zero.
# [ ] No integer overflow - don't pre-guess limits.
# [ ] Raise sys.setrecursionlimit for deep DFS.
# [ ] 2D list: [[0]*m for _ in range(n)], never [[0]*m]*n.
# [ ] Don't use a mutable default argument: def f(a=[]) is a bug.
# [ ] Reading all input with .read().split() handles newlines/spaces.
# [ ] Division by zero raises; guard it.
# [ ] `x in list` is O(n); use a set/dict when it matters.
# [ ] Copy a list with a[:] or list(a), not b = a.

if __name__ == "__main__":
    # Small self-demo that needs no input.
    huge = int("1" * 5000)
    print("digits:", len(str(huge)))
    print("floor div:", huge // (10 ** 2500) == int("1" * 2500))
    print("mod ok:", huge % (10 ** 2500) == int("1" * 2500))
    print("split ok:", big_split(huge, 1) is not None)


sys.set_int_max_str_digits(1000000)
```

notes/sort_lambda.cpp — 17 lines

```cpp
#include<bits/stdc++.h>


using namespace std;
int main()
{
    array<int, 10> arr = { 1,2,3,4,5,6,7,8,9 };

    sort(begin(arr),
         end(arr),
         [](int a, int b) {return a%2 > b%2; });

    for (auto item : arr)
      cout << item << " ";

    return 0;
}
```